

Comparing Binary Classifiers

XIE YUJIE, G2604541A
IN6227-2026-Assignment-1
Variant-2

Model name & version: GPT-6 Astra

LLM interface used: ChatUI

Repository link: [https://github.com/xyjpang/NTU-](https://github.com/xyjpang/NTU-assignments/blob/5ce9c93a8a3c82bb5415a0242b7d2e744f79539b/IN6227%20DATA%20MINING/assignment%201/S)

[assignments/blob/5ce9c93a8a3c82bb5415a0242b7d2e744f79539b/IN6227%20DATA%20MINING/assignment%201/S](https://github.com/xyjpang/NTU-assignments/blob/5ce9c93a8a3c82bb5415a0242b7d2e744f79539b/IN6227%20DATA%20MINING/assignment%201/S)
KILL.md

INTRODUCTION

This experiment compares logistic regression and random forest for predicting the binary label yes or no from 15 tabular predictors. The supplied training and test files contain 31,112 and 13,334 records, respectively. The predictors comprise seven numerical variables, including `aptitude_score` and `load_ratio`, and eight categorical variables, including `region` and `mineral_type`. Because the input folder does not explain the label or feature provenance, the task is treated as a statistical classification problem without assigning a real-world meaning to the outcome.

Among labelled training records, 7,464 are yes and 23,645 are no; the positive class therefore represents 23.99%. This imbalance makes accuracy alone insufficient for comparing models. The experiment uses positive-class F1 as its main criterion and examines precision, recall and ROC AUC to explain the trade-offs. A regularized linear model provides a simple baseline, while a forest of decision trees allows nonlinear relationships and interactions.

METHODS OR PROCEDURES

Data exploration and cleaning. Each file was checked for feature types, missing values, class frequencies, duplicates, extreme values and schema consistency. There were 47 missing predictor cells in training and 20 in test. Three training records and one test record lacked a target. The three unlabelled training records were excluded; the unlabelled test record received predictions but was excluded from evaluation. This left 31,109 training records and 13,333 evaluation records, including 3,168 positive test cases (23.76%).

No exact duplicate records, duplicate feature vectors, conflicting labels for identical vectors or identical feature vectors across the two sets were found. Numerical values contained no infinities, and categorical values had no surrounding whitespace. All observed test categories occurred in training. Interquartile-range screening flagged extreme values, especially `activity_duration` (5,350 training records) and `load_ratio` (2,527). These were retained because statistical

extremeness alone does not establish an error, and no valid domain bounds were supplied.

Feature preparation. Numerical missing values were replaced with training medians. Missing categorical values were represented by an explicit category; existing Unknown and Other values were retained. One-hot encoding avoided imposing an artificial order on categories. Numerical predictors were standardized for logistic regression, while random forest used their original scales. All 15 predictors were retained, producing 90 columns after preprocessing on the full training set. No additional feature selection or domain-based engineering was justified by the available information.

Validation design. The supplied test split was preserved. Both models used the same shuffled, stratified three-fold cross-validation within the labelled training set, with random seed 42. Imputation, encoding and scaling were fitted separately inside each training fold to prevent leakage. Candidate settings were ranked by mean validation F1. The chosen pipelines were then refitted on all labelled training records. Both used balanced class weights and a fixed probability threshold of 0.5; neither settings nor thresholds were selected using test performance.

Model training. Logistic regression used L2 regularization, with C selected from 0.1, 1 and 10 [1]. Cross-validation selected C=0.1, giving mean F1 of 0.6597 (fold standard deviation 0.0044). The final model converged in 76 iterations, below the 2,000-iteration limit. Random forest used 200 trees and compared maximum depths of 12 and unrestricted depth, with minimum leaf sizes of 1 or 5 [2]. The selected forest had unrestricted depth and five samples per leaf, with mean validation F1 of 0.6647 (standard deviation 0.0032). It stopped after the fixed 200 trees. Neither model produced training warnings.

RESULTS

Table 1 reports performance on the same 13,333 labelled test records. F1 combines positive-class precision and recall through their harmonic mean [3]. ROC AUC evaluates the ranking of positive and negative examples

across thresholds. Always predicting no would achieve 76.24% accuracy on this test set but would identify no positive cases; this arithmetic reference was not trained as an additional model.

Table 1. Test performance (yes is the positive class).

Metric	Logistic regression	Random forest
Accuracy	78.33%	80.60%
F1	0.6488	0.6630
Precision	52.76%	56.44%
Recall	84.22%	80.33%
ROC AUC	0.8879	0.8897

Random forest improved F1 by 0.0143 and accuracy by 2.27 percentage points over logistic regression. Precision increased from 52.76% to 56.44%, while recall fell from 84.22% to 80.33%. The forest correctly classified 10,746 records, compared with 10,444 for logistic regression.

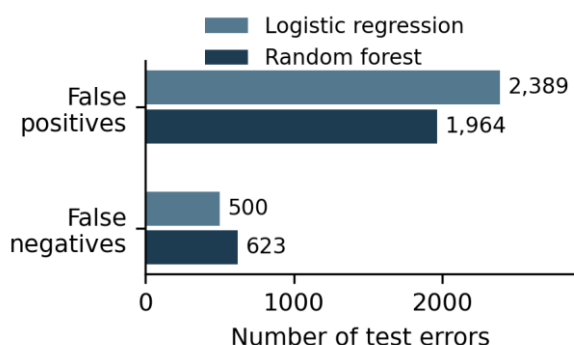


Figure 1. Error counts at probability threshold 0.5.

Figure 1 makes the trade-off explicit. The forest made 425 fewer false-positive predictions, but missed 123 additional positive cases. Its confusion matrix contained 8,201 true negatives and 2,545 true positives; the corresponding counts for logistic regression were 7,776 and 2,668.

DISCUSSION

The forest gives the strongest overall result under the prespecified F1 objective. Its improvement is mainly associated with higher precision: a larger fraction of predicted positives is correct. However, the two ROC AUC values differ by only 0.0018, indicating similar ranking performance. The test results support a modest practical advantage for the forest at the chosen threshold, rather than a large separation between the methods.

The choice also depends on the consequences of errors. Logistic regression detects 84.22% of positive cases, compared with 80.33% for the forest, so it would be preferable if avoiding missed positives were the overriding aim. Its linear structure is also easier to inspect. The forest accommodates interactions and nonlinear effects, but its combined decision process is harder to summarize. Since no application-specific error

costs were provided, the comparison follows the agreed F1 criterion.

Both models still produce many false positives: even the better precision is only 56.44%. Balanced class weights give minority examples greater influence during training, but do not remove uncertainty or guarantee calibrated probabilities. A future study could examine probability calibration and alternative decision thresholds using a separate training validation procedure. Those changes should be evaluated on fresh held-out data rather than chosen after inspecting the present test results.

These findings are limited to one supplied train/test split and two model families with small parameter grids. Cross-validation variation is reported, but no formal significance test establishes that the observed F1 difference will persist on new samples. Undocumented feature semantics and sampling procedures prevent causal interpretation or a claim of real-world deployment readiness. More informative metadata and independent data would help determine whether the observed relationships generalize.

CONCLUSION

Random forest is the preferred model for this experiment because it achieves the higher positive-class F1 (0.6630) and accuracy (80.60%) on the supplied test set. Logistic regression remains competitive and offers higher recall. The comparison shows why several metrics are needed for imbalanced classification: fewer false alarms can come at the cost of more missed positives. Training-only preprocessing and model selection provide a reproducible basis for this conclusion, while the error counts and data limitations define its scope.

REFERENCES

- [1] scikit-learn. LogisticRegression. Version 1.8 documentation. Online documentation.
- [2] scikit-learn. RandomForestClassifier. Version 1.8 documentation. Online documentation.
- [3] scikit-learn. f1_score. Version 1.8 documentation. Online documentation.

REFLECTION

1. Human oversight

I remained involved throughout the execution of the skill rather than accepting the generated output directly. I set up two checkpoints so I could check whether the AI was executing the process correctly.

First checkpoint, after the initial data exploration, I reviewed the identified target variable, missing-value patterns, class distribution, proposed preprocessing steps, evaluation metric, and candidate classification models. I checked whether these decisions were reasonable for the characteristics of the dataset before allowing the workflow to continue.

Second checkpoint, after training, I also reviewed the model comparison. I examined the performance metrics and the difference between the training/cross-validation results and the final test results to check for possible overfitting. Where necessary, I adjusted the skill instructions or asked the agent to reconsider a decision. For example, I verified the train-test sample sizes and confirmed that the split followed the intended 80/20 ratio while preserving the class distribution through stratification.

If I ran it again, I would improve the skill by asking it to generate and compare several suitable classification models first, and then rank or recommend them based on their suitability for the dataset. I could then review the options and manually select the two models to be used for the final comparison.

2. Critical evaluation

One decision made by the skill that I found worth questioning was the use of balanced class weights for both classification models. The skill applied balanced class weights because the positive class represented only about 24% of the training data. I think this was a reasonable approach to class imbalance, but I do not think it should have been applied automatically.

The final results showed relatively high recall but much lower precision, especially for the positive class. This suggests that giving more weight to minority-class observations may have encouraged the models to predict the positive class more frequently, resulting in more false-positive predictions. Therefore, I would have preferred the skill to compare both the default and balanced class-weight settings during cross-validation and select between them according to the primary F1 metric.

I agree that balanced class weights were a defensible choice for this experiment, but they should ideally be treated as a hyperparameter to be evaluated rather than a fixed preprocessing decision. In a future version of the skill, I would also consider tuning the classification threshold on the training validation data to better control the trade-off between precision and recall.

3. Trustworthiness

To verify that the generated report was accurate and reliable, I manually checked one of the reported evaluation results rather than relying entirely on the AI-generated output. Specifically, I verified the random forest confusion matrix and its reported accuracy. The confusion matrix contained 8,201 true negatives, 2,545 true positives, 1,964 false positives, and 623 false negatives, which together sum to 13,333 test records. The number of correct predictions was therefore $8,201 + 2,545 = 10,746$, giving an accuracy of approximately 80.60%, which matched the value reported in the final report.

I also checked that the test set was kept separate from model selection and hyperparameter tuning. According to the workflow, preprocessing and cross-validation were performed using the training data only, while the supplied test set was used only for the final evaluation. This reduced the risk of data leakage and gave me greater confidence that the reported test performance was reliable.